PROJECT MANAGEMENT

UNIT-1

[CS-604]

Syllabus -

Conventional Software management. Evolution of software economics. Improving software economics : reducing product size, software processes, team effectiveness, automation through software environments. Principles of modern software managements.

PROJECT MANAGEMENT

1. SOFTWARE PROJECT MANAGEMENT (SPM)

Software Project management is the process of planning, organizing, scheduling & controlling Software projects to achieve goals within time, cost & quality.

Key Points:-

- Temporary activity (has start & end)
- Uses resources (time, money, people)
- Goal - Oriented.

- Needs proper planning.

2. CONVENTIONAL SOFTWARE MANAGEMENT

Conventional Software management is the traditional (old) way of managing software projects where everything is planned at the beginning & followed strictly till the end.

Key features:-

- Fixed Planning at start
- Sequential process (step)
- Less flexibility
- Late testing
- No proper risk handling.

Problems:

- Late delivery
- Budget overrun
- Poor quality
- No risk management.

Evolution Of Software Economics

[May 2023, June 2025, Dec 2024, May 2023]

Software economics deals with cost, time, effort & resources required to develop software.

The Evolution of software economics shows how software development has changed from slow, costly & inefficient to fast cost-effective & high-quality.

1. Early Stage (Traditional Era)

Features:

- Manual coding (no advanced tools)
- Sequential development (waterfall Model)
- Testing done at the end.
- No proper planning.

Problems:

- High Cost
- Long development time
- Poor quality
- High failure rate.

⇒ Software projects were difficult to manage and often unsuccessful.

2. Transition Stage (Need for Improvement)

Reason for change:

- Increasing software size & complexity.
- Demand for faster delivery.
- Need for better quality.
- High competetion in industry.

⇒ This created the need for better management techniques & tools.

3. Modern Stage (After Evolution)

Improvements:

- Use of modern models (Agile, Iterative)
- Automation tools for coding & testing
- Continuous integration & testing.
- Better planning & scheduling
- Reusable components.

Results:

- Faster development
- Reduced cost
- Improved quality
- Better customer satisfaction

Three Generation of Software Evolution

Feature	Conventional (1960s-1970s)	Transition (1980s-1990s)	modern (2000s-Present)
Approach	Custom built	Software Engineering	Software Production
Tools	Custom, primitive tool	Repeatable, Off-the shelf Tool	Integrated, automated environments.
Components	100% custom	70% custom 30% Commercial	30% custom 70% off the shelf.
Predictability	Poor	moderate / Inconsistent	usually on budget & schedule

IMPROVING SOFTWARE ECONOMICS

[June 2025, May 2023] [May 2024]

Improving Software Economic means reducing cost, reducing time & increasing productivity in software development.

In simple words:

Do more work in less time with less cost & better quality.

Objective of Improvement

- Reduce development cost
- Reduce development time
- Improve software quality
- Increases productivity
- Better resource utilization

Ways to Improve Software Economics

1. Reducing Product Size

If software size is smaller → development becomes easier.

Explanation :-

Large software requires more coding, testing and maintainance.

If we reduce unnecessary features, we can save time & cost.

How to do:

- Remove unwanted features
- Divide Project into modules.
- Use reusable components.

Result/Impact:

- less cost
- Faster development
- Easy maintenance.

2. Improving Development Process

Concept:

A good development process helps complete work in a structured and efficient way.

Explanation:

Old methods (like Waterfall) were rigid and did not allow changes.

Modern methods (like Agile) allow continuous improvement & flexibility.

Techniques:

- Agile development
- Iterative model (develop step by step)
- Continuous testing & feedback.

Impact:

- Early detection of errors
- Faster delivery
- Better quality software

3. Improving Team Skills (Personnel).

Concept:

Skilled team members can complete tasks faster and more accurately.

Explanation:

An experienced developer understands problems quickly & writes better code.

Unskilled developers make more mistakes, increasing cost & time.

Techniques:

- Training Programs
- Hiring experienced developers
- Improving communication

Impact:

- Higher Productivity
- Fewer errors
- Faster Problem solving

4. Use of Better tools & Technology

Concept:

Tools help automate tasks & reduce manual work.

Explanation:

Earlier developers wrote everything manually. Now tool helps in coding, testing, debugging & version control.

Examples:

- IDE (VS Code, IntelliJ)
- Version Control (Git)
- Testing Tools

Impact:

- Faster development
- Improved accuracy
- Better quality Software.

5. Reusability

Concept:

Reusing existing code instead of writing from scratch.

Explanation:

Developers often needs similar functionality in different projects.

Instead of writing it again, they can reuse libraries & modules.

Example:

- Libraries
- Framework
- APIs

Impact:

- Saves time
- Reduce Development cost
- Improves reliability.

6. Automation:

Concept:

Using tools to perform repetitive task automatically.

Explanation:

Tasks like testing, building & deployment can be automated.

This reduce human effort & increases speed.

Examples:

- Automated Testing tools
- Continuous Integration (CI/CD)

Impact:

- Faster Process
- Fewer human errors
- Consistent result.

7) Risk Management

Concept:

Identifying & handling risks before they become problems.

Explanation:

Every Project has risks like delays, technical issues or budget problems.

If identified early, they can be controlled.

Types of Risk –

- Technical Risk
- Cost Risk
- Schedule Risk

Impact -

- Avoid project failure
- Better decision making
- Smooth project execution

8. Better Planning & Scheduling

Concept:

Proper planning helps in organizing work efficiently.

Explanation:

Without planning, work becomes unorganized & delayed.

Scheduling ensures tasks are completed on time.

Techniques:

- Grantt charts
- Task scheduling
- milestones.

Impact:

- On-time delivery
- Better resource utilization
- Reduced delays

Summary

Software Economics improves when:

- Product size is reduced
- Process is improved
- Team is skilled
- Tools & automation are used
- Risk are managed
- Planning is Proper

PRINCIPLES OF MODERN SOFTWARE MANAGEMENT

[May 2024, Dec 2024, June 2025, May 2023]

Modern software management focuses on building software faster, better & with less cost while handling real-world changes.

1. Architecture First Approach

Start with a strong system design (architecture) before coding.

Explanation:

- Just like building a house needs a blueprint, software also needs a plan.
- Decide structure, modules, technologies early.

Importance:

- Reduce future Problems
- makes system scalable & maintainable.

Example:

Before building a shopping app, you decide:

- Payment system
- User authentication
- Product database structure.

2. Iterative Development

Software is developed in small cycles (iterations) instead of building everything at once.

Explanation:

Each Iteration includes:

1. Planning
2. Designing
3. Coding
4. Testing

Then the cycle repeats.

Importance:

- Early versions are available quickly
- Feedback can be taken from users
- Errors can be fixed early.

Example:

- Iteration - 1 → Login system
- Iteration - 2 → Add products
- Iteration - 3 → Payment system.

3. Continuous Integration & Testing

Code is integrated (combined) frequently & tested regularly.

Explanation:

- Developers work on different parts
- Code is merged into a central system daily.
- Automated tools test the code.

Importance:

- Detects bugs early
- Avoids last-minute errors
- maintains system stability.

Example:

- Every time a developer uploads code, tests run automatically.

4. Component Based Development

Software is built using pre-made reusable components.

Explanation:

- Components are independent modules
- Can be reused in multiple projects

Importance:

- Saves development time
- Reduces cost
- Improves reliability

Example:

- Using a ready-made login module.
- Using payment gateway APIs.

5. Change Management

Handling changes in requirements in a controll-ed way.

Explanation:

Requirements often change during development.

All changes are:

- Recorded
- Analyzed
- Approved before implementation

Tools used:

Version control System (like Git)

Importance:

- Avoids confusion
- Keeps project organized.

6. Risk Management

Identifying & handling possible problem early

Explanation:

Steps:

1. Identifying risks
2. Analyze impact
3. Plan solution
4. monitor continuously.

Types of risk

- Technical risk
- Schedule risk
- Cost risk

Importance:

- Prevents project failure
- Reduces uncertainty.

7. Team Collaboration & Communication

All team members work together effectively.

Explanation:

- Regular meetings
- Clear communication
- Sharing updates.

Importance:

- Reduces misunderstanding
- Improve efficiency.

8. Automation

Using tools to automate repetitive tasks.

Explanation:

Automation can be used for

- Testing
- Building software
- Deployment.

Importance:

- Saves time
- Reduce human error
- Speeds up development